ANIMESH MUDIT

Backend Engineer & Systems Developer | Full-Stack Engineering | Applied AI & Distributed Systems

Kanpur, India • github.com/animeshmudit • linkedin.com/in/animeshmudit • animeshmudit@gmail.com

PROFILE

Computer Science undergraduate (B.Tech, PSIT, Kanpur; CGPA 7.59/10) building production-oriented full-stack and applied-AI systems. Strongest work sits at the intersection of backend engineering, application architecture, LLM orchestration, retrieval systems, databases, payments, and security. Experienced end-to-end — from product requirements through frontend, backend, persistence, integrations, and operational concerns — with a security-minded approach to state ownership, authentication, and third-party integrations.

TECHNICAL SKILLS

Languages: Python, C++, Java, JavaScript, TypeScript, SQL, Bash, HTML, CSS

Backend & APIs: FastAPI, Flask, Streamlit, Pydantic, RESTful API design, asynchronous I/O (asyncio), authentication, server-side business logic

Frontend: React, Next.js, Vite, Tailwind CSS, Framer Motion, Three.js, WebGL

Databases & Storage: PostgreSQL/Supabase, MySQL, SQLite, MongoDB, Firebase/Firestore, Oracle Database 21c XE

AI Systems & Retrieval: Retrieval-Augmented Generation (RAG), FAISS, Hugging Face / SentenceTransformers embeddings, Groq API (with local-model fallback routing), Llama-based & Qwen2.5-Coder models, Ollama, LM Studio, FastFlowLM, multi-agent pipelines, prompt orchestration

Engineering & Systems: Git/GitHub, Docker & Docker Compose, Pytest, uv package manager, Linux (CachyOS/Arch — kernel-level tuning, btop, fastfetch), webhooks, idempotency, row-level security (RLS), IDOR prevention, HMAC verification

EDUCATION

B.Tech — Computer Science & Engineering

Pranveer Singh Institute of Technology (PSIT), Kanpur | 2024–2028 | CGPA: 7.59/10

Relevant coursework: Data Structures & Algorithms, Object-Oriented Analysis & Design, Database Management Systems (DBMS), Operating Systems, Computer Networks.

SELECTED PROJECTS & SYSTEM ARCHITECTURES

RAAHAT — Emotionally Aligned Conversational AI (Hybrid RAG)

Python • FastAPI • Groq API • Llama-based models • FAISS • Hugging Face / SentenceTransformers • Supabase • MongoDB • Telegram • Docker Compose

- Designed as an emotionally supportive conversational AI — explicitly not a therapy, diagnosis, or medical assistant.
- Built a multi-tiered RAG pipeline over curated CBT/ACT/DBT, WHO stress, cognitive-distortion, grounding, and psychological-first-aid material, converted into dense vector embeddings and indexed via FAISS.
- Separated safety from generation using deterministic crisis detection and hyperbole handling, so the LLM is never the sole safety gate; crisis responses use a backend-controlled Indian resource card rather than model-invented helpline information.
- Implemented shared conversational persistence across web and Telegram clients, with onboarding and session handling.
- Built an asynchronous FastAPI middleware layer managing dynamic session isolation and persistent dialogue history across MongoDB and Supabase.
- Used multi-query semantic/metadata-aware retrieval with normalized embeddings, keyword expansion, deduplication, and structured prompt construction before generation.
- Added a resilient high-availability fallback: a circuit-breaker reroutes inference to local models (Ollama) automatically when Groq is rate-limited or degraded.
- Centralized orchestration in a dedicated “brain” and knowledge layer, with evaluation/testing support and containerized deployment via Docker Compose.

ARCHITECTURE

- **Clients:** Web, Telegram, CLI
- **API / Orchestration:** FastAPI → brain.py → session/memory flow → async middleware for session isolation
- **Safety:** Deterministic crisis detection, hyperbole filtering, fixed backend-controlled resource response

- **Knowledge pipeline:** PDF ingestion → chunking → embeddings → normalized vectors → FAISS index
- **Retrieval:** Keyword generation → multi-phrase expansion → semantic/metadata search → deduplication
- **Generation:** Retrieved context + conversation history → prompt orchestration → Groq LLM, with Ollama circuit-breaker fallback

SETU — AI-Assisted Emergency Response & Geospatial Disaster Relief Platform

FastAPI • Python • Firebase/Firestore • Geocoding/Maps APIs • AI Severity Analysis • Real-Time Communication • Google Solution Challenge

- Architected backend services to coordinate emergency incident intake, resource dispatch, and response during active disaster scenarios, using FastAPI as the service boundary.
- Designed around rapid incident intake, AI-assisted severity assessment, location-aware handling, and real-time communication, with AI treated as one component in the response pipeline rather than the entire system.
- Developed a geospatial volunteer-matching algorithm using Haversine distance vectors and radial bounding queries to prioritize field responders within critical survival timeframes.
- Engineered real-time Firestore client synchronization with optimistic state updates to preserve mission-critical dispatch tracking under degraded network connectivity.

ARCHITECTURE

- **Incident intake:** Client → FastAPI → validation / incident creation
- **Intelligence:** AI severity analysis → prioritization signal
- **Dispatch:** Geospatial volunteer matching (Haversine + radial bounding queries) → responder prioritization
- **Response state:** Firestore document streams → optimistic client sync → geolocation/maps → real-time communication

Crafty Mind Studio — Full-Stack E-Commerce Platform

Next.js 15 • React 19 • Supabase/PostgreSQL • Razorpay • Resend • Tailwind CSS

- Built a complete commerce workflow spanning storefront, cart/checkout, orders, payments, transactional email, admin operations, and analytics.
- Implemented server-authoritative pricing so the client cannot determine the payable amount.
- Hardened payment processing with HMAC-verified webhooks and idempotent reconciliation to prevent duplicate processing and inconsistent order states.
- Used Supabase/PostgreSQL for persistent data with row-level security (RLS) and session-aware access control.
- Designed sensitive state transitions to be owned by the backend rather than trusted to browser-side checks.
- Built an admin console and operational views for managing commerce data and monitoring activity.

ARCHITECTURE

- **Presentation:** Next.js/React storefront, Tailwind UI
- **Application:** Server-side routes/actions → business rules → order/payment orchestration
- **Data:** Supabase PostgreSQL, RLS, session-aware access
- **Payment:** Client checkout → server-authoritative amount → Razorpay → signed webhook → idempotent reconciliation
- **Order state:** Backend owns order/payment transitions; client cannot finalize payment state
- **Operations:** Admin console, analytics, transactional email via Resend

RepoGuard — Autonomous Multi-Agent Static Application Security Auditor (SAST)

Python • Local LLMs (Qwen2.5-Coder) • FastFlowLM/Ollama • Pydantic

- Orchestrated a multi-agent pipeline executing AST parsing and localized context injection via low-latency local LLM runtimes (Ollama/FastFlowLM), governed by Pydantic JSON schemas.
- Engineered a deterministic multi-agent SAST tool for targeted code auditing and vulnerability patching without cloud API exposure or IP leaks.
- Implemented strict Pydantic schema validation across agent boundaries (Auditor Agent → Patch Synthesizer → Verification Agent) to eliminate hallucinated syntax and guarantee compile-ready remediation.
- Optimized local quantized model parameters (Qwen2.5-Coder 7B) with tailored prompt templates and streaming inference, reducing per-file audit turnaround by 40%.

ARCHITECTURE

- **Pipeline:** AST parsing → localized context injection → local LLM inference (Ollama/FastFlowLM)
- **Agent boundary:** Auditor Agent → Patch Synthesizer → Verification Agent, each governed by Pydantic schemas
- **Deployment:** Fully local inference — no cloud API exposure or IP leakage

The Huddle — Academic Collaboration & Team-Formation Platform (Sept–Oct 2025)

Python • Flask • SQLite • SQLAlchemy ORM • RESTful API

- Collaborative product built during a September–October 2025 team engineering project, covering end-to-end product development and application engineering.
- Engineered a modular relational REST API to manage high-cardinality student team formation, skill discovery, and academic collaboration workflows.
- Designed a third-normal-form (3NF) relational schema with composite indices and foreign-key cascading constraints to ensure transactional integrity.
- Integrated structured input validation, granular role-based access control, and endpoint pagination to maintain sub-50ms API response latency.

ARCHITECTURE

- **Pattern:** Clean layered MVC architecture with repository-pattern abstraction
- **Data model:** 3NF relational schema, composite indices, foreign-key cascades
- **API:** Role-based access control, input validation, paginated endpoints, index-optimized query pipelines

Interactive Personal Engineering Portfolio (3D/WebGL)

React • TypeScript • Vite • Framer Motion • Three.js • WebGL • Lenis • CSS Shaders

- Built an interactive, industrial retro-styled portfolio intended to demonstrate engineering depth through the interface itself rather than a static project list.
- Features dynamic 3D silicon-die wireframes and an interactive CRT monitor model with real-time scanline post-processing, alongside architecture/code visualizations to communicate technical work.
- Implemented a GPU-accelerated client-side rendering pipeline with custom fragment shaders for phosphor CRT rastering and decoupled tick loops.
- Optimized rendering to a steady 60 FPS via geometry instancing, frustum culling, and non-blocking Lenis smooth-scroll physics.
- Applied responsive visual QA across desktop and mobile viewports and removed generic/debug-like filler elements in favor of deliberate product presentation.

ARCHITECTURE

- **UI:** React + TypeScript + Vite
- **Motion:** Framer Motion + Lenis smooth scroll
- **3D / visual:** Three.js, WebGL buffers, custom fragment shaders, interactive technical visualizations

Modular News Ingestion Engine

Python • NewsAPI • Structured Logging • OOP Design Patterns

- Implemented an object-oriented Python ETL ingestion service to query, filter, and normalize multi-source live news feeds.
- Used an extensible producer-consumer ingestion model with factory patterns and structured JSON logging pipelines, including robust rate-limit backoff logic.

ARCHITECTURE

- **Model:** Producer-consumer ingestion → factory-pattern source handlers → structured JSON logging

ENGINEERING STRENGTHS

- Backend ownership: APIs, persistence, business rules, state transitions, and third-party integrations.
- Applied AI: retrieval, embeddings, multi-agent orchestration, memory, safety boundaries, fallbacks, and evaluation.
- Security-minded development: IDOR prevention, server-authoritative values, HMAC verification, idempotency, session management, and row-level security.
- Full-stack execution: frontend interaction through backend logic, database state, integrations, and deployment concerns.

SYSTEMS & ALGORITHMIC PRACTICE

- Data structures & algorithms: continuous practice in C++ across LeetCode and HackerRank, focused on graph theory, dynamic programming, and tree traversal.
- Linux systems: dual-boot CachyOS power user — kernel customization, package orchestration via uv/Pacman, automated network tethering (gnirehtet), and terminal telemetry diagnostics (btop, fastfetch).
- Engineering practices: decoupled modular architectures, Git version control, semantic commit protocols, and automated environment isolation.

CERTIFICATIONS

- Infosys Springboard — HTML / CSS / Bootstrap

- Infosys Springboard — Python
- KnowledgeGate — Java

TECHNICAL INTERESTS

Backend systems • AI/LLM applications • retrieval systems • application security • payment systems • developer tooling • product engineering • interactive web experiences

---

Master-resume note: this document is intentionally architecture-heavy and consolidates two prior source resumes. It is the source from which shorter, ATS- and role-specific versions should be generated without losing the underlying engineering story.